Chapter 1

Embedded Software Design A — Software Engineering

ENEL712 Embedded Systems Design — Lecture 8

Summary

Software engineering applied to embedded systems: the software lifecycle, requirements, abstraction and modelling (UML, FSMs), and testing.

Topics Covered

- Fundamentals of software engineering
- Software process activities
- Requirements engineering
- System modelling and abstraction
- Software testing

1.1 Fundamentals of Software Engineering

Software engineering covers all aspects of software production, from specification to maintenance. In modern embedded systems, software often dominates development cost and is more expensive to maintain than to build.

1.2 Software Process Activities

- Specification: define what the software must do and its constraints, with the customer.
- Development: design and program the software.
- Validation: confirm the software meets customer requirements.
- Evolution: modify the software to reflect market or customer change.

1.3 Requirements Engineering

1.3.1 Functional vs non-functional

Type	Describes
Functional	Services the system provides, reactions to inputs, and behaviour in particular situations.
Non-functional	System properties and constraints: timing, reliability, response time, storage, etc.

About 56 % of software defects originate in requirements. Fixing a scope change post-launch can cost up to 200× more than during initial analysis.

1.4 System Modelling and Abstraction

- Abstraction: ignore some properties to focus on the most important — top-down (add detail) or bottom-up (combine solved parts).
- UML: graphical notation for representing different system views to aid analyst/customer communication.
- State machine models: states as nodes, events as arcs; superstates expand into sub-state diagrams for complex behaviour.

1.5 Software Testing

Test type	Goal
Validation testing	Show the system meets requirements; every requirement should have $\geq$1 test.
Defect testing	Make the system perform incorrectly to expose bugs.

Testing reveals the presence of errors, but cannot prove their absence.

1.6 Use Cases & Applications

- Medical/automotive/aerospace embedded products legally must follow software-engineering processes (IEC 62304, ISO 26262, DO-178C).
- Even one-person hobby projects benefit from short specifications and state diagrams — they catch contradictions before code is written.
- Test-driven development on PC code can run in CI even if the final binary targets an MCU; mock hardware modules behind interfaces.

1.7 Worked Example: A Tiny Specification

Spec: 'door access controller'. Functional and non-functional requirements look like this:

- FR1: When a valid card is presented, the controller shall unlock the door for 5 s.
- FR2: After 3 invalid attempts within 60 s, the controller shall lock out for 5 minutes.

- FR3: All access events shall be logged to flash with a timestamp.
- NFR1: Card-to-unlock latency shall be < 500 ms.
- NFR2: Battery backup shall sustain operation for ≥ 24 h.

Each requirement maps to one or more tests (validation tests) and a state in the FSM. Defect testing then attacks the edges: bad card data, near-simultaneous reads, brown-out during a write.

1.8 State Diagram in Words

Sketch of the door controller as a hierarchical FSM:

- States: LOCKED, UNLOCKING, UNLOCKED, LOCKOUT.
- Events: VALID_CARD, INVALID_CARD, TIMER_EXPIRES.
- Transitions: LOCKED + VALID_CARD → UNLOCKING (start motor); UNLOCKING + TIMER_EXPIRES → UNLOCKED; UNLOCKED + TIMER_EXPIRES → LOCKED; LOCKED + 3×INVALID_CARD → LOCKOUT.
- Superstate ANY + brown-out → SAFE (all locks engaged, log, halt).

1.9 Cost-of-Defect Curve

Industry data shows the cost of fixing a defect rises roughly an order of magnitude per phase: ~1× in requirements, ~5–10× in design, ~10–40× in code, ~100× in test, ~200× post-launch. Catching ambiguity in requirements is therefore by far the highest leverage activity.

Key Definitions

Software Engineering
Discipline covering all aspects of software production from spec to maintenance.

Abstraction
Ignoring some properties to focus on the essential parts of a problem.

Functional Requirement
Service or behaviour the system must provide.

Non-Functional Requirement
Constraint or property such as timing, reliability, or storage.

Validation Testing
Tests intended to demonstrate that the software meets its requirements.

Defect Testing
Tests intended to expose incorrect behaviour or bugs.

Requirement Traceability
Practice of linking every test back to one or more requirements (and vice versa).

Verification vs Validation
Verification: 'are we building the product right?' Validation: 'are we building the right product?'

Mock / Stub
: Stand-in module that simulates a hardware peripheral or external service for desk-side testing.

Sources

- Lecture 8.pdf